

A Word Is a Vector

Word embeddings: numbers for words, and what addition, subtraction and the dot product do with them.

A word as a list of numbers

```
E50 has shape (50000, 50): 50,000 words, and 50 numbers for each word
the first ten words: ['the', 'of', 'to', 'and', 'in', 'a', 'for', 'that', 'on', 'is']
Your group's word is 'piano'. It is in row 4,516, and its vector is:
[-0.67  0.73 -1.22 -0.83 -0.15  0.95  0.09  0.71 -1.21  1.33  0.43  0.56
 -0.39  0.89 -0.22 -1.11 -0.62  0.59  0.1  -1.33  0.92  0.54 -0.48  0.29
  0.73 -0.03 -0.83 -0.34 -0.71 -1.17  2.59 -0.88  0.79  0.65  0.71  0.25
  1.3   1.01  0.81  0.62  0.37 -0.13 -0.41 -0.59  0.17  0.6   0.96 -1.16
 -1.37  0.49]
```

- A **word embedding** gives every word a vector of m numbers. Nobody chose them by hand: a program read six billion words and gave similar vectors to words that occur in similar surroundings.
- These are **GloVe** vectors, computed at Stanford from Wikipedia and news text. The course keeps the 50,000 most frequent words, with 50 numbers for each word and with 300.
- A matrix of the same kind as `x` in Act 1: a row is one word, a column is one dimension, and a column has no name.

Which words are close to a word?

```
def nearest(word, U, k=8):  
    """The k words whose vectors point most nearly in the direction of the vector of word."""  
    similarity = U @ U[row(word)]          # one dot product for every word  
    order = np.argsort(-similarity)        # row numbers, from the most similar word on  
    return [(words[i], round(float(similarity[i]), 2)) for i in order[1:k + 1]]    # order[0] is the word itself
```

The words closest to 'piano', with their cosine similarity:

violin	0.93
cello	0.92
flute	0.85
concerto	0.84

- Two vectors are compared by the angle between them: the **cosine similarity** is their dot product after each is divided by its length. The **dot product** is the sum of the products of matching entries.
- `U @ v` computes the dot product of **every row** of `U` with `v`: 50,000 dot products in one operation.

Three more words

```
for word in ["newark", "python", "happy"]:  
    print(word, "->", [other for other, similarity in nearest(word, U50)])
```

```
newark -> ['albany', 'camden', 'trenton', 'minneapolis', 'wilmington', 'bridgeport', 'laguardia',  
'hartford']  
python -> ['php', 'owl', 'mouse', 'perl', 'monkey', 'monty', 'scripting', 'rhino']  
happy -> ['everyone', 'everybody', 'really', 'me', 'definitely', 'maybe', 'feel', 'i']
```

A word has one vector, whatever its number of meanings: the words closest to *python* are programming languages, snakes and a group of comedians.

Arithmetic on words

```
def analogy(a, b, c, U, k=3, leave_out=True):  
    """a is to b as c is to which word? The k words closest to: vector of b - vector of a + vector of c."""  
    target = U[row(b)] - U[row(a)] + U[row(c)]  
    similarity = U @ (target / np.linalg.norm(target))  
    if leave_out:  
        similarity[[row(a), row(b), row(c)]] = -2    # the three words of the question cannot be the answer  
    return [words[i] for i in np.argsort(-similarity)[:k]]
```

```
man : king = woman : ?      ['queen', 'throne', 'prince']  
the same, nothing left out: ['king', 'queen', 'throne']  
france : paris = italy : ?  ['rome', 'milan', 'turin']  
walk : walked = play : ?    ['played', 'playing', 'players']  
cold : colder = warm : ?    ['warmer', 'drier', 'cooler']
```

How many numbers for each word?

E50 has shape (50000, 50): 2,500,000 numbers, 10 megabytes

E300 has shape (50000, 300): 15,000,000 numbers, 60 megabytes

Correct answers out of 40: with 50 numbers for each word 29, with 300 numbers 35

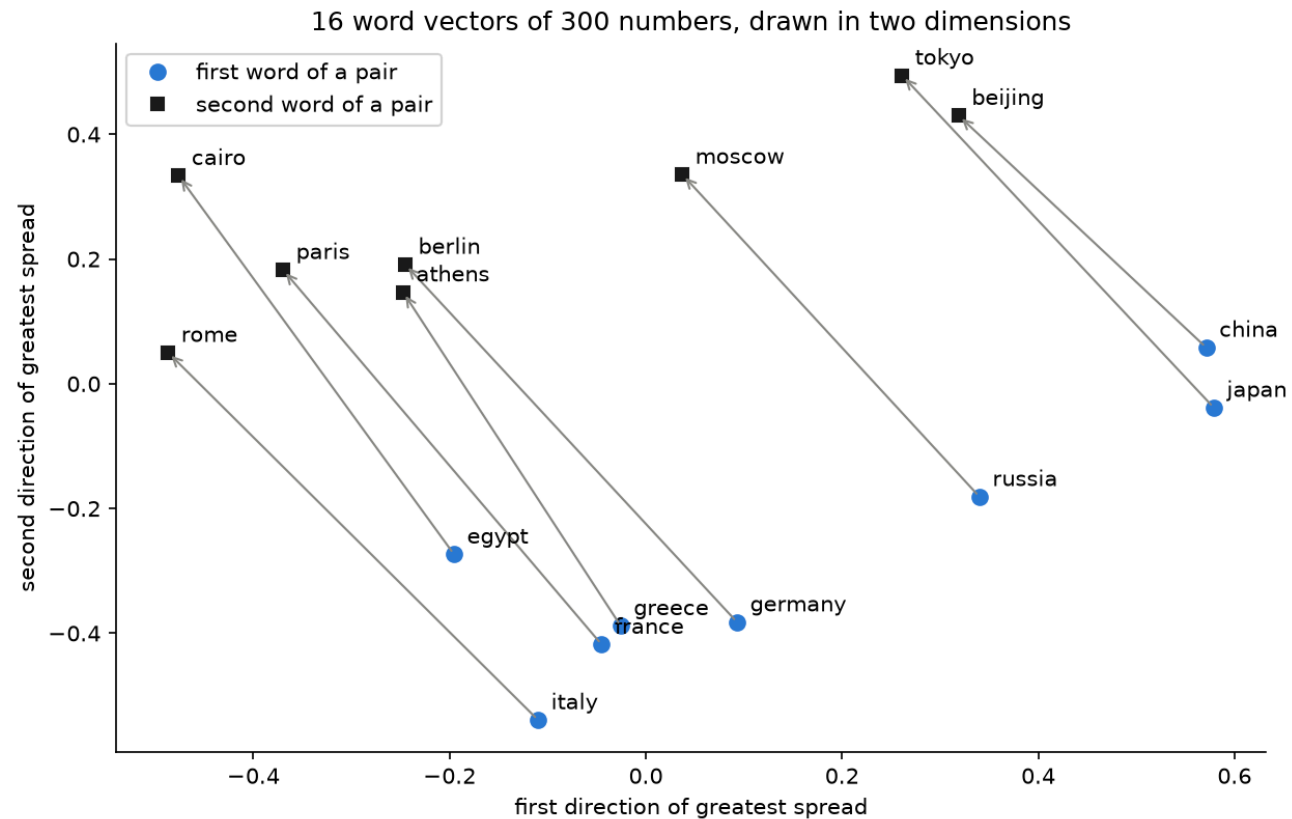
The questions on which the two versions give different answers:

question	expected	50 numbers	300 numbers
germany : berlin = spain : ?	madrid	rome	madrid
england : london = austria : ?	vienna	stockholm	vienna
canada : ottawa = peru : ?	lima	lima	peruvian
man : woman = uncle : ?	aunt	daughter	aunt
long : longer = short : ?	shorter	usually	shorter
eat : ate = write : ?	wrote	writing	wrote
child : children = man : ?	men	who	men
mouse : mice = foot : ?	feet	weight	feet

50 numbers : comparing one word with all 50,000 words takes 0.23 milliseconds

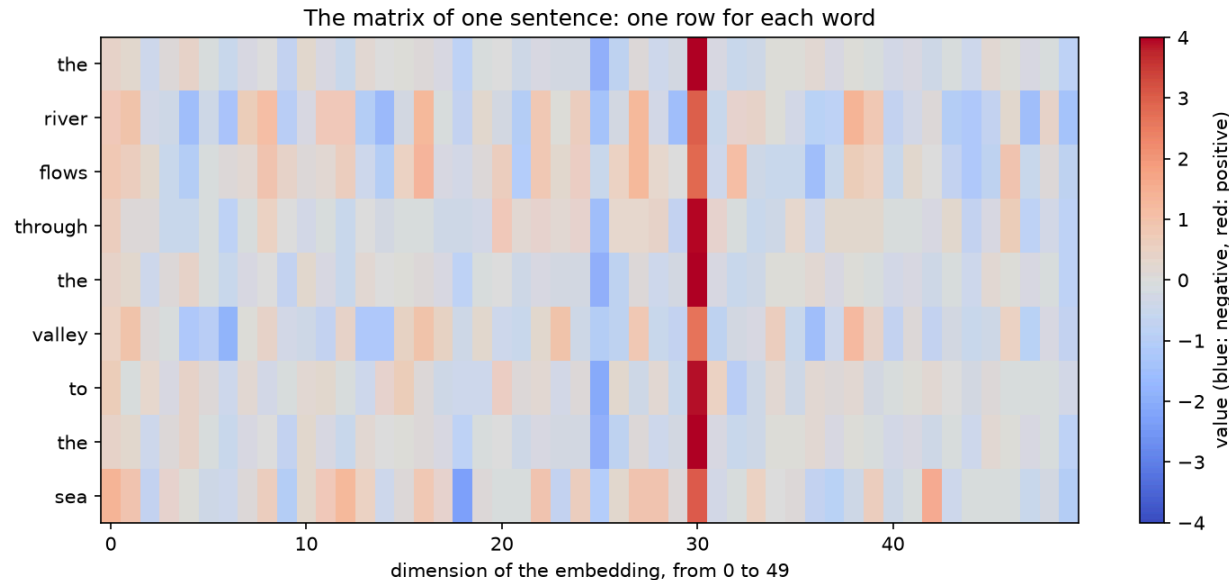
300 numbers : comparing one word with all 50,000 words takes 0.84 milliseconds

A picture of word vectors



Three hundred numbers cannot be drawn. `np.linalg.svd` finds the two directions along which these sixteen vectors are most spread out, and each word is drawn at its two coordinates. The arrows are nearly parallel: the difference between a country and its capital is nearly the same vector for every pair.

A text is a sequence of vectors



'the dog chased the cat' and 'the cat chased the dog':

rows 0 and 3 of the first matrix, both the word 'the', are equal: True

the two matrices are equal: False

the averages of their rows are equal: True

- A text of L words is a matrix with L rows, **in the order of the words**. The same words in another order give another matrix, with the same average row: an operation that ignores the order cannot tell the two sentences apart.

A batch of texts

shape of T: (3, 9, 50) - text, position in the text, dimension of the embedding
text 0: 9 words and 0 rows of zeros: 'the river flows through the valley to the sea'
text 1: 3 words and 6 rows of zeros: 'students write code'
text 2: 8 words and 1 rows of zeros: 'she scored two goals in the final match'
numbers in the batch: 1,350

Data	One point	A batch
a row of a table	m numbers	$n \times m$
a colour image	$H \times W \times 3$	$n \times H \times W \times 3$
a text of L words	$L \times m$	$B \times L \times m$, padded with rows of zeros

Language models receive text in this form: a batch of sequences of vectors.

Now in Colab: Act 3

- **Act 3** of the class notebook: the words closest to your group's word, and three analogy questions of your own. Find one that the vectors answer well and one that they answer badly.
- Draw a picture of your own: five to eight pairs of words that share one relation.
- **Post**, as a reply to your group's first post: the picture, the relation you chose, whether the arrows came out nearly parallel, and your best question with its answer. One member posts; begin with the names of all the members.

What we saw today

Data	One point	A batch
a row of a table	m numbers	$n \times m$
a grey image	$H \times W$	$n \times H \times W$
a colour image	$H \times W \times 3$	$n \times H \times W \times 3$
a text of L words	$L \times m$	$B \times L \times m$

- An operation on a whole array runs inside NumPy. The same loop in the interpreter is tens of times slower, and a loop over a NumPy array slower still.
- The dot product, addition and subtraction of vectors say something even about words.
- **After class:** the full notebooks of the three acts, linked on the meeting's page, and Review Quiz 10, which opens now.